

条款 32：确定你的 public 继承塑模出 is-a 关系

Make sure public inheritance models "is-a."

在《*Some Must Watch While Some Must Sleep*》(W. H. Freeman and Company, 1974) 这本书中，作者 William Dement 说了一个小故事，谈到他曾经试图让学生记下课程中最重要的教导。书上说，他告诉他的班级，一般英国学生对于发生在 1066 年的黑斯廷斯 (Hastings) 战役所知不多。如果有学生记得多一些，Dement 强调，无非也只是记得 1066 这个数字而已。然后 Dement 继续其课程，其中只有少数重要信息，包括“安眠药反而造成失眠症”这类有趣的事情。他一再要求学生，纵使忘了课程中的其他每一件事，也要记住这些数量不多的事情。Dement 在整个学期中不断耳提面命这样的话。

课程结束后，期末考的最后一道题目是：“写下你从本课程获得的一件永生不忘的事”。当 Dement 批改试卷，他目瞪口呆。几乎每一个人都写下“1066”。

这就是为什么现在我要戒慎恐惧地对你声明，以 C++ 进行面向对象编程，最重要的一个规则是：public inheritance (公开继承) 意味 “is-a” (是一种) 的关系。把这个规则牢牢地烙印在你的心中吧！

如果你令 class D (“Derived”) 以 public 形式继承 class B (“Base”)，你便是告诉 C++ 编译器（以及你的代码读者）说，每一个类型为 D 的对象同时也是一个类型为 B 的对象，反之不成立。你的意思是 B 比 D 表现出更一般化的概念，而 D 比 B 表现出更特殊化的概念。你主张“B 对象可派上用场的任何地方，D 对象一样可以派上用场”（译注：此即所谓 Liskov Substitution Principle），因为每一个 D 对象都是一种（是一个）B 对象。反之如果你需要一个 D 对象，B 对象无法效劳，因为虽然每个 D 对象都是一个 B 对象，反之并不成立。

C++ 对于“public 继承”严格奉行上述见解。考虑以下例子：

```
class Person { ... };  
class Student: public Person { ... };
```

根据生活经验我们知道，每个学生都是人，但并非每个人都是学生。这便是这个继承体系的主张。我们预期，对人可以成立的每一件事——例如每个人都有生日——对学生也都成立。但我们并不预期对学生可成立的每一件事——例如他或她

注册于某所学校——对人也成立。人的概念比学生更一般化，学生是人的一种特殊形式。

于是，承上所述，在 C++ 领域中，任何函数如果期望获得一个类型为 **Person**（或 **pointer-to-Person** 或 **reference-to-Person**）的实参，都愿意接受一个 **Student** 对象（或 **pointer-to-Student** 或 **reference-to-Student**）：

```
void eat(const Person& p);      //任何人都会吃
void study(const Student& s);   //只有学生才到校学习
Person p;                      //p 是人
Student s;                     //s 是学生
eat(p);                        //没问题，p 是人
eat(s);                        //没问题，s 是学生，而学生也是 (is-a) 人
study(s);                      //没问题，s 是个学生
study(p);                      //错误！p 不是个学生
```

这个论点只对 **public** 继承才成立。只有当 **Student** 以 **public** 形式继承 **Person**，C++ 的行为才会如我所描述。**private** 继承的意义与此完全不同（见条款 39），至于 **protected** 继承，那是一种其意义至今仍然困惑我的东西。

public 继承和 **is-a** 之间的等价关系听起来颇为简单，但有时候你的直觉可能会误导你。举个例子，企鹅（**penguin**）是一种鸟，这是事实。鸟可以飞，这也是事实。如果我们天真地以 C++ 描述这层关系，结果如下：

```
class Bird {
public:
    virtual void fly();        //鸟可以飞
    ...
};

class Penguin: public Bird {   //企鹅是一种鸟
    ...
};
```

突然间我们遇上了乱流，因为这个继承体系说企鹅可以飞，而我们知道那不是真的。怎么回事？

在这个例子中，我们成了不严谨语言（英语）下的牺牲品。当我们说鸟会飞的时候，我们真正的意思并不是说所有的鸟都会飞，我们要说的只是一般的鸟都有飞行能力。如果谨慎一点，我们应该承认一个事实：有数种鸟不会飞。我们来到以下

继承关系，它塑模出较佳的真实性：

```
class Bird {  
    ...                               //没有声明 fly 函数  
};  
  
class FlyingBird: public Bird {  
public:  
    virtual void fly();  
    ...  
};  
  
class Penguin: public Bird {  
    ...                               //没有声明 fly 函数  
};
```

这样的继承体系比原先的设计更能忠实反映我们真正的意思。

即便如此，此刻我们仍然未能完全处理好这些鸟事，因为对某些软件系统而言，可能不需要区分会飞的鸟和不会飞的鸟。如果你的程序忙着处理鸟喙和鸟翅，完全不在乎飞行，原先的“双 classes 继承体系”或许就相当令人满足了。这反映出一个事实，世界上并不存在一个“适用于所有软件”的完美设计。所谓最佳设计，取决于系统希望做什么事，包括现在与未来。如果你的程序对飞行一无所知，而且也不打算未来对飞行“有所知”，那么不去区分会飞的鸟和不会飞的鸟，不失为一个完美而有效的设计。实际上它可能比“对两者做出区隔”更受欢迎，因为这样的区隔在你企图塑模的世界中并不存在。

另有一种思想派别处理我所谓“所有的鸟都会飞，企鹅是鸟，但是企鹅不会飞，喔欧”的问题，就是为企鹅重新定义 fly 函数，令它产生一个运行期错误：

```
void error(const std::string& msg); //定义于另外某处  
  
class Penguin: public Bird {  
public:  
    virtual void fly() { error("Attempt to make a penguin fly!"); }  
    ...  
};
```

很重要的是,你必须认知这里所说的某些东西可能和你所想的的不同。这里并不是说“企鹅不会飞”,而是说“企鹅会飞,但尝试那么做是一种错误”。

如何描述其间的差异?从错误被侦测出来的时间点观之,“企鹅不会飞”这一限制可由编译期强制实施,但若违反“企鹅尝试飞行,是一种错误”这一条规则,只有运行期才能检测出来。

为了表现“企鹅不会飞,就这样”的限制,你不可以为 Penguin 定义 fly 函数:

```
class Bird {  
    ...                               //没有声明 fly 函数  
};  
  
class Penguin: public Bird {  
    ...                               //没有声明 fly 函数  
};
```

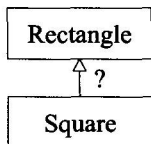
现在,如果你试图让企鹅飞,编译器会对你的背信加以谴责:

```
Penguin p;  
p.fly();                               //错误!
```

这和采取“令程序于运行期发生错误”的解法极为不同。若以那种做法,编译器不会对 p.fly 调用式发出任何抱怨。条款 18 说过:好的接口可以防止无效的代码通过编译,因此你应该宁可采取“在编译期拒绝企鹅飞行”的设计,而不是“只在运行期才能侦测它们”的设计。

或许你承认你对鸟类缺乏直觉,但基础几何学得不错。喔,是吗?那么我请问,正方形和矩形之间可能有多么复杂?

好,请回答这个简单的问题: class Square 应该以 public 形式继承 class Rectangle 吗?



“咄！”你说，“当然应该如此！每个人都知道正方形是一种矩形，反之则不一定”，这是真理，至少学校是这么教的。但是我不认为我们还在象牙塔内。

考虑这段代码：

```
class Rectangle {
public:
    virtual void setHeight(int newHeight);
    virtual void setWidth(int newWidth);
    virtual int height() const;      //返回当前值
    virtual int width() const;
    ...
};

void makeBigger(Rectangle& r)      //这个函数用以增加 r 的面积
{
    int oldHeight = r.height();
    r.setWidth(r.width() + 10);    //为 r 的宽度加 10
    assert(r.height() == oldHeight); //判断 r 的高度是否未曾改变
}
```

显然，上述的 `assert` 结果永远为真。因为 `makeBigger` 只改变 `r` 的宽度；`r` 的高度从未被更改。

现在考虑这段代码，其中使用 `public` 继承，允许正方形被视为一种矩形：

```
class Square: public Rectangle { ... };
Square s;
...
assert(s.width() == s.height()); //这对所有正方形一定为真。
makeBigger(s);                  //由于继承，s 是一种 (is-a) 矩形，
                                //所以我们可以增加其面积。
assert(s.width() == s.height()); //对所有正方形应该仍然为真。
```

这也很明显，第二个 `assert` 结果也应该永远为真。因为根据定义，正方形的宽度和其高度相同。

但现在我们遇上了一个问题。我们如何调解下面各个 `assert` 判断式：

- 调用 `makeBigger` 之前，`s` 的高度和宽度相同；
- 在 `makeBigger` 函数内，`s` 的宽度改变，但高度不变；

- `makeBigger` 返回之后, `s` 的高度再度和其宽度相同。(注意 `s` 是以 *by reference* 方式传给 `makeBigger`, 所以 `makeBigger` 修改的是 `s` 自身, 不是 `s` 的副本。)

怎么样?

欢迎来到“public 继承”的精彩世界。你在其他领域(包括数学)学习而得的直觉, 在这里恐怕无法如预期般地帮助你。本例的根本困难是, 某些可施行于矩形身上的事情(例如宽度可独立于其高度被外界修改)却不可施行于正方形身上(宽度总是应该和高度一样)。但是 public 继承主张, 能够施行于 base class 对象身上的每件事情, 每件事情稍, 也可以施行于 derived class 对象身上。在正方形和矩形例子中(另一个类似例子是条款 38 的 sets 和 lists), 那样的主张无法保持, 所以以 public 继承塑模它们之间的关系并不正确。编译器会让你通过, 但是一如我们所见, 这并不保证程序的行为正确。就像每一位程序员一定学过的(某些人也许比其他入更常学到): 代码通过编译并不表示就可以正确运作。

不要因为你发展经年的软件直觉在与面向对象观念打交道的过程中失去效用, 便心慌意乱起来。那些知识还是有价值的, 但现在你已经为你的“设计”军械库加上继承(inheritance)这门大炮, 你也必须为你的直觉添加新的洞察力, 以便引导你适当运用“继承”这一支神兵利器。当有一天有人展示一个长达数页的函数给你看, 你终将回忆起“令 Penguin 继承 Bird, 或是令 Square 继承 Rectangle”的概念和趣味; 这样的继承有可能接近事实真象, 但也有可能不。

is-a 并非是唯一存在于 classes 之间的关系。另两个常见的关系是 **has-a** (有一个) 和 **is-implemented-in-terms-of** (根据某物实现出)。这些关系将在条款 38 和 39 讨论。将上述这些重要的相互关系中的任何一个误塑为 **is-a** 而造成的错误设计, 在 C++ 中并不罕见, 所以你应该确定你确实了解这些个“classes 相互关系”之间的差异, 并知道如何在 C++ 中最好地塑造它们。

请记住

- “public 继承”意味 **is-a**。适用于 base classes 身上的每一件事情一定也适用于 derived classes 身上, 因为每一个 derived class 对象也都是一个 base class 对象。

条款 33：避免遮掩继承而来的名称

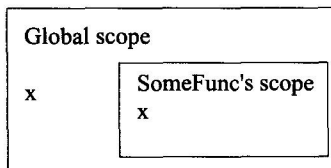
Avoid hiding inherited names.

关于“名称”，莎士比亚说过这样一句话：“名称是什么呢？”他问，“一朵玫瑰叫任何名字还是一样芬芳。”吟游诗人也写过这样的话：“偷了我的好名字的人呀……害我变得好可怜。”完全正确。这把我们引到了 C++ “继承而来的名称”。

这个题材和继承其实无关，而是和作用域（scopes）有关。我们都知道在诸如这般的代码中：

```
int x;                      //global 变量
void someFunc()
{
    double x;              //local 变量
    std::cin >> x;         //读一个新值赋予 local 变量 x
}
```

这个读取数据的语句指涉的是 local 变量 x，而不是 global 变量 x，因为内层作用域的名称会遮掩（遮蔽）外围作用域的名称。我们可以这样看本例的作用域形势：



当编译器处于 `someFunc` 的作用域内并遭遇名称 `x` 时，它在 local 作用域内查找是否有东西带着这个名称。如果找到就不再找其他作用域。本例的 `someFunc` 的 `x` 是 `double` 类型而 `global x` 是 `int` 类型，但那不要紧。C++ 的名称遮掩规则（name-hiding rules）所做的唯一事情就是：遮掩名称。至于名称是否应和相同或不同的类型，并不重要。本例中一个名为 `x` 的 `double` 遮掩了一个名为 `x` 的 `int`。

现在导入继承。我们知道，当位于一个 `derived class` 成员函数内指涉（refer to）`base class` 内的某物（也许是个成员函数、`typedef`、或成员变量）时，编译器可以找出我们所指涉的东西，因为 `derived classes` 继承了声明于 `base classes` 内的所有东西。实际运作方式是，`derived class` 作用域被嵌套在 `base class` 作用域内，像这样：